

Τεκμηρίωση Βάσης Δεδομένων Νοσοκομείου

Ιωάννης Τζέιμς Μακμίλλαν, 03123438
Κατερέλος Χαράλαμπος, 03122626

ΕΙΣΑΓΩΓΗ ΚΑΙ ΣΚΟΠΟΣ

Το έγγραφο αυτό απόσκοπει στη διευκόλυνση του αναγνώστη, ώστε να εξοικειωθεί με τη δομή της βάσης δεδομένων και να κατανοήσει τη λογική πίσω από τον σχεδιασμό της. Η αναφορά χωρίζεται σε τρία βασικά μέρη.

Αρχικά, παρουσιάζεται ο ορισμός των βασικών οντοτήτων, οι σχέσεις μεταξύ τους, καθώς και οι κανόνες και οι περιορισμοί που διέπουν τη δημιουργία τους. Στη συνέχεια, περιγράφεται ή διαδικασία φόρτωσης της βάσης με κατάλληλα δεδομένα, τα οποία έχουν ως στόχο να αναδείξουν τη λειτουργικότητα του συστήματος και να επιτρέψουν την εκτέλεση των ζητούμενων ερωτημάτων.

Τέλος, παρουσιάζονται τα SQL ερωτήματα που υλοποιήθηκαν, μαζί με σύντομη επεξήγηση της λογικής τους και των αποτελεσμάτων που επιστρέφουν.

ΔΟΜΗ ΤΟΥ CODEBASE

Το codebase αποτελείται από τέσσερα κύρια μέρη:

- install.db, από όπου μπορεί να ληφθεί το schema, τα triggers και τα procedures της βάσης
- load.db το οποίο φορτώνει με την καταλληλη σειρά λόγω dependency, τα δεδομένα στην βάση
- Code όπου περιέχει όλα τα παραπάνω χωρισμένα ανα feature που απαιτείται για την βάση. Πιο συγκεκριμένα, κάθε feature αποτελείται από: :
 - install.db αρχείο για την εγκατάσταση των κατάλληλων tables του feature (χρήσιμο για modularity και readability)
 - load.db ή και load folder για την φόρτωση των δεδομένων
 - triggers folder όπου εφαρμόζονται τα business rules αυτού του feature
 - procedures για την εφαρμογή των use cases της βάσης. Ενώ στην πραγματικότητα ή βάση θα πρεπε να περιέχει όλα τα δυνατά CRUD operations για την ορθότητα και πληρότητα της, κατι τετοιο πιστευουμε δεν ήταν απόλυτα αναγκαίο για την συγκεκριμένη εργασία και αρκεστήκαμε στην δημιουργία procedures που έχουν να κάνουν είτε με την αλλαγή είτε με την προσθήκη δεδομένων μέσω ορισμενων procedures που διασφαλίζουν σε συνεργασία με τα triggers την φόρτωση και ύπαρξη δεδομένων στο συστημα που ακολουθούν τις οδηγίες τις ασκήσεις και την λογική του συστήματος.

ΠΕΡΙΓΡΑΦΗ ΧΑΡΑΚΤΗΡΙΣΤΙΚΩΝ (FEATURES)

Να σημειωθεί ότι ή εφαρμογή χωρίστηκε σε features αυθαίρετα αλλά εν γένει με γνώμονα όσο περισσότερο γίνεται την ανεξαρτησία ύπαρξης του σε σχέση με τα υπόλοιπα.

STAFF (staff/install.db)

ΣΧΕΤΙΚΑ TABLES : iatros, proswpiko, nosileutis, dioikitiko Το staff δηλαδή ιατροί, νοσηλευτές και διοικητικό είναι το πρώτο κομμάτι της εφαρμογής και το πιο εύκολο να οριστεί καθώς ή δημιουργία τους δεν εξαρτάται από κάποια άλλη οντότητα. Κοινό στοιχείο

- Και τα τρία tables αποτελούν υποοντότητα του table προσωπικού το οποίο με το field typos_proswpikou δείχνει την ιδιότητα της κάθε εγγραφής στο προσωπικό. Το ίδιο το προσωπικό είναι και αυτό μια υποοντότητα του

table anthropos, το οποίο αν και έχει περισσότερο ως θεωρητική οντότητα στο ER χρησιμοποιήθηκε και στην πράξη, ώστε ύστερα να μοιραστεί τα ίδια χαρακτηριστικά και ο ασθενής.

- primary key (amka)

LOOKUP TABLES

- vathmida_iatrou: Για τον iatros table συγκεκριμένα για την βαθμίδα χρησιμοποιήθηκε look up table για μεγαλύτερη ευελιξία.

TRIGGERS (code/staff/triggers)

Εδώ ή μονη οντότητα στην οποία έπρεπε να εφαρμοστούν ειδικοί κανόνες, πέρα από κανόνες και περιορισμοί ακεραιότητας φυσικά που εφαρμόζει το ίδιο το schema με την χρήση primary, unique είναι ο ιατρός. Συγκεκριμένα, εφαρμόσαμε insert trigger με timing before στον ιατρο ώστε να διασφαλιστούν τα παρακάτω :

- Η βαθμίδα ιατρού υπάρχει

```
SELECT COUNT(*)
INTO grade_exists
FROM vathmida_iatrou
WHERE vathmida_id= NEW.vathmida;

IF grade_exists = 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Η βαθμίδα του γιατρού δεν υπάρχει';
END IF;
```

- Η βαθμίδα του χρειάζεται επόπτη και αν χρειάζεται και δεν έχει περάσει τότε ή εγγραφή να αποτύχει ή αν δεν μπορεί να έχει επόπτη και έχει

```
IF NEW.amka_epoptis IS NULL THEN
IF has_to_be_supervised = 1 THEN
SIGNAL SQLSTATE '45000'

SET MESSAGE_TEXT = 'Ο γιατρός με αυτήν την βαθμίδα πρέπει αναγκαστικά να
έχει επόπτη';
END IF;

-- Case 2: doctor has supervisor
ELSE
IF has_to_be_supervised = 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Αυτός ο γιατρός με αυτήν την βαθμίδα δεν μπορεί να έχει
επόπτη';
END IF;
```

- απαγορεύει την κυκλική εποπτεία και την εποπτεία του ίδιου του του εαυτού

```
IF epoptis_tou_epopti = NEW.amka THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Δεν επιτρέπεται κυκλική εξάρτηση εποπτείας';
END IF;
```

```

IF NEW.amka_epoptis = NEW.amka THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Ο γιατρός δεν μπορεί να είναι επόπτης του εαυτού του';
END IF;

```

Για την επαλήθευση ορθότητας ύπαρξης ιατρού στο σύστημα, δίνεται και αρχείο (staff/tests/doctor_tests.sql) όπου τεστάρει την απάντηση του συστήματος σε εγγραφή έγκυρων και μη δεδομένων.

PROCEDURES

Add_iatros, add_dioikitiko, add_nosileutis με τις κατάλληλες παραμέτρους ώστε να διευκολύνει την ταυτόχρονη εισαγωγή σε 3 διαφορετικά tables.

TMIMA (departments/install.db)

Σχετικά tables : proswpiko_anikei_se_tmima: (amka, tmima_id), tmima, klini

Το proswpiko_anikei_se_tmima δημιουργήθηκε για να υποστηρίζει την ιδιότητα των ιατρών να ανήκουν σε παραπάνω από ένα τμήματα παρ'όλα αυτά πρέπει να απαγορεύει την εισαγωγή νοσηλευτές/ών ή διοικητικού σε πολλαπλά τμήματα ενώ επίσης απαγορεύει έναν διευθυντή να ανήκει σε πολλαπλά τμήματα (departments/triggers/department_triggers.sql)

```

(before insert on proswpiko_anikei_se_tmima)
IF katigoria IN ('Νοσηλευτής', 'Διοικητικό') THEN

SELECT COUNT(*)
INTO anikei_idi
FROM proswpiko_anikei_se_tmima
WHERE amka_proswpikou = NEW.amka_proswpikou;

IF anikei_idi = 1 THEN
SET msg = CONCAT('Το προσωπικό τύπου ',

```

katigoria, ' ανήκει ήδη σε τμήμα. Αν θες να αλλάξεις το τμήμα κάνε CALL το

```

update_tmima_proswpikou');
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = msg;
END IF;

```

Τα triggers για το τμήμα αφορούν before insert και διασφαλίζουν ότι ο διευθυντής του τμήματος είναι όντως διευθυντής

```

SELECT v.vathmida_onoma
INTO vathmida_dieftinti
FROM iatros i
join vathmida_iatrou v on v.vathmida_id = i.vathmida
WHERE amka = NEW.amka_dieftinti;

IF vathmida_dieftinti != 'Διευθυντής' THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Ο ιατρός με αυτό το AMKA δεν έχει βαθμίδα διευθυντή';
END IF;

SELECT COUNT(*)
INTO dieftinti_se_allo_tmima
FROM tmima
WHERE amka_dieftinti = NEW.amka_dieftinti;

IF dieftinti_se_allo_tmima != 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Ο διευθυντής αυτός είναι ήδη διευθυντής σε άλλο τμήμα. Αν

```

θες να τον ορίσεις διευθυντή αυτού του τμήματος κάνε INSERT το τμήμα χωρίς διευθυντή

```

και μετά χρησιμοποίησε την update_tmima_proswpikou';
END IF;

```

Η klini table δεν έχει καποίο trigger παρ' όλα αυτά αλλάζει ή κατάσταση της μέσω trigger στην νοσηλευτέςία όπως θα δουμε ύστερα. Το identifier της είναι ένας αριθμός και έχει δύο πιθανές καταστάσεις.

ICD/KEN/IATRIKESPRAXEIS (code/icd, code/ken) Σχετικά tables: icd, ken, iatrikespraxeis Αφορούν tables τα οποία πρέπει να γεμισουν με συγκεκριμένα δεδομένα από CSV. Δεν απαιτούν καποίον περιορισμό για την εισαγωγή τους

ASTHENEIS (code/asthenis)

Σχετικά tables: asthenis, ektakti_epafi Για την οντότητα του ασθενή γίνεται ή υπόθεση ότι μπαίνει στο σύστημα ανεξαρτήτως αν παραπέμπεται για νοσηλευτέςία ή όχι (θεωρείται ασθενής και όταν μπαίνει στο τμήμα επειγόντων περιστατικών, δεν έχουμε καποίο table visitor). ο ασθενής επίσης συνδέεται μέσω foreign key με το table του anthropos

EFIMERIA (code/shifts)

Σχετικά tables: efimeria, efimeria_proswpiko

LOOKUP TABLES(code/shifts/install.sql)

```
Vardia(3 rows πρωινή, απόγευματινή, νυχτερινή), efimeria_requirements
+-----+-----+-----+-----+
+-----+-----+-----+-----+
| vardia_id | vardia_onoma          | vardia_ora_ekkinisis | vardia_ora_lixis |
+-----+-----+-----+-----+
endiamesi_ora_anapausis_hours |
```

epitreptes_sinexomenes_vardies |

```
+-----+-----+-----+-----+
+-----+-----+-----+-----+
|      1 | Πρωινή          | 07: 00: 00          | 15: 00: 00          |
8 |      2 | Απόγευματινή    | 15: 00: 00          | 23: 00: 00          |
8 |      3 | Νυχτερινή       | 23: 00: 00          | 07: 00: 00          |
8 |      3 |                 | 3 |                 |
+-----+-----+-----+-----+
+-----+-----+-----+-----+
```

Όπως για για το vathmida_iatrou, τα business rules μας αναγκάζουν να εφαρμόσουμε περιορισμούς με βάση τις ώρες ανάπαυσης και τις επιτρεπτές συνεχόμενες αδειες, με τα ζητούμενα της εργασίας να αποτελούν παραμέτρους του κανόνα (π.χ. ΚΚανένα μέλος προσωπικού δεν μπορεί να συμμετέχει σε περισσότερες από 3 συνεχόμενες νυχτερινές βάρδιες.)

---+

SCHEMA(code/shifts/install.sql)

Η efimeria table περιέχει την βάρδια του κάθε τμήματος

(tmima, imerominia, vardia) ως primary key ενώ το efimeria_proswpiko κάνει

reference στην εφημερία και περιέχει (amka, tmima, imerominia, vardia) ως primary key (π.χ. 3 ιατροί, 6 νοσηλευτές, 2 διοικ σε μια βάρδια αρα τουλάχιστον 11 efimeria_proswpiko objects reference efimeria) Για να εφαρμοστεί constraint το οποίο αναφέρεται σε πολλά valid insertions όπως το παραδειγμα πιο πάνω δημιουργήθηκε ένα field statusEf στην efimeria για το οποίο γίνεται check κάθε φορά μέσω του trigger after insert

on efimeria με την χρήση βοηθητικής συνάρτησης efimeria_check που

παίρνει ως ορίσματα (tmima, vardia, imerominia)->0 or 1.

TRIGGERS

RULE 1

Το σύστημα επιβάλλει τα ακόλουθα μέγιστα όρια βαρδιών ανά μήνα ανά κατηγορία προσωπικού: Ιατροί 15, Νοσηλευτές 20 και Διοικητικό Προσωπικό 25.

```
SELECT COALESCE(COUNT(*), 0)
INTO v_monthly_count
FROM efimeria_proswpiko e
WHERE e.amka_proswpiko = NEW.amka_proswpiko
AND YEAR(e.imerominia) = YEAR(NEW.imerominia)
AND MONTH(e.imerominia) = MONTH(NEW.imerominia);

IF v_monthly_count >= v_max_allowed THEN
set msg = 'Έχει ξεπεράσει το μηνιαίο όριο εφημεριών';
call add_error(msg);
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = msg;
END IF;
```

RULE 2

Επιπλέον, μεταξύ δύο διαδοχικών βαρδιών του ίδιου μέλους προσωπικού πρέπει να μεσολαβεί ελάχιστο διάστημα ανάπαυσης 8 ωρών.

```
IF v_last_shift_end IS NOT NULL THEN
SET v_hours_since_last =
TIMESTAMPDIFF(HOUR, v_last_shift_end, v_new_shift_start);

IF v_hours_since_last < v_rest_hours THEN
call add_error('Δεν υπάρχει το ελάχιστο διάστημα ανάπαυσης μεταξύ των
βαρδιών');
SIGNAL SQLSTATE '45000'

SET MESSAGE_TEXT = 'Δεν υπάρχει το ελάχιστο διάστημα ανάπαυσης μεταξύ των
βαρδιών';
END IF;
END IF;
```

RULE 3

Κανένα μέλος προσωπικού δεν μπορεί να συμμετέχει σε περισσότερες από 3 συνεχόμενες νυχτερινές βάρδιες.

```
IF v_allowed_consecutive IS NOT NULL THEN
SELECT COUNT(*)
INTO v_consecutive_same_type
FROM efimeria_proswpiko e
JOIN vardia v
ON v.vardia_id = e.vardia
WHERE e.amka_proswpiko = NEW.amka_proswpiko
AND e.vardia = NEW.vardia
AND e.imerominia IN (DATE_SUB(NEW.imerominia, INTERVAL 1 DAY),
DATE_SUB(NEW.imerominia, INTERVAL 2 DAY),
DATE_SUB(NEW.imerominia, INTERVAL 3 DAY));

IF v_consecutive_same_type >= v_allowed_consecutive THEN
SET msg = CONCAT('Έχει ξεπεράσει το όριο συνεχόμενων νυχτερινών βαρδιών για AMKA ',
NEW.amka_proswpiko);
call add_error(msg);
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = msg;
END IF;
```

Κάθε εφημερία τμήματος πρέπει να καλύπτεται από τουλάχιστον τρεις ιατρούς, έξι νοσηλευτές/εστές και δύο διοικητικούς υπαλλήλους. Επιπλέον, σε κάθε βάρδια που συμμετέχει ειδικευόμενος ιατρός, πρέπει υποχρεωτικά να είναι παρών τουλάχιστον ένας ιατρός βαθμίδας Επιμελητής Α΄ ή Διευθυντής.

FOR EACH ROW

TESTS (shifts/test)

Για το τεστ χρησιμοποιήθηκε AI όπου του ζητήθηκε να χρησιμοποιήσει την συνάρτησή μας `add shift` για όλα τα `edge cases` που μας ενδιαφέρουν οπότε για φορές που πρέπει να αποτύχει το `shift` δεν απόθηκεύτηκε ενώ για τις φορές που πέτυχε μπορούμε να το βρούμε

TRIAGE (code/triage)

ΣΧΕΤΙΚΑ TABLES efimeria se kathikon triage, parapobi gia nosileia.dialogistoi xeiwn

ΣΧΕΤΙΚΑ VIEWS : oura anamenomenwn

Το τμήμα επειγόντων περιστατικών αν και δεν δηλώνεται ρητά το αντιμετωπίσαμε ως ένα κανονικό τμήμα, δηλαδή οι κανόνες των εφημεριών για την ύπαρξη χ γιατρων κλπ ισχυει ακριβως. Η μονη διαφορά είναι οτι δημιουργουμε το efimeria_se_kathikon_triage το οποιο αποθηκευει το (nosileutis_id, tmima, imerominiamvardia) όπου φυσικά το τμήμα ειναι fixed id (στην συγκεκριμένη περιπτωση 20) Ετσι κατα την εισαγωγή dialogistoixeiwn

```
CREATE PROCEDURE register_dialogi(IN p_amka astheni CHAR(11),
```

```
IN p_wra_afiksis    DATETIME, IN p_symptomata    TEXT, IN p_epipedo    TINYINT, IN
p_apotelesma    VARCHAR(20), IN p_odigies    TEXT, IN p_wra_oloklirosis DATETIME)
```

Δεν χρειάζεται να πέρασουμε καπόιον νοσηλευτέρευτη καθώς εκείνος είναι ήδη δηλωμένος αναγκαστικά εκείνη την ώρα με βάση το οπότε γίνεται ή διαλογη αλλιως μπλοκάρει

```
SELECT e.amka_proswpiko
INTO v_amka_nosilevti
FROM efimeria_se_kathikon_triage e
JOIN vardia v
ON v.vardia_id = e.vardia
WHERE p_wra_afiksis >= TIMESTAMP(
DATE(e.imerominia), v.vardia_ora_ekkinisis)
AND p_wra_afiksis < CASE
WHEN v.vardia_ora_lixis > v.vardia_ora_ekkinisis THEN
TIMESTAMP(
DATE(e.imerominia), v.vardia_ora_lixis)
ELSE
```

```

TIMESTAMP(DATE(e.imerominia) + INTERVAL 1 DAY, v.vardia_ora_lixis)
END

```

ORDER BY RAND()

```

LIMIT 1;
IF v_amka_nosilevti IS NULL THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'No nosileutis found for register_dialogi';
END IF;

```

Το παραπάνω εφαρμόζεται και στο ίδιο το trigger

RULE 1(CODE/TRIAGE/QUEUE_VIEW.SQL)

Οι ασθενείς εξυπηρετούνται με βάση το επίπεδο επείγοντος και, για το ίδιο επίπεδο, ακολουθείται αυστηρή σειρά προτεραιότητας βάσει χρόνου άφιξης (FIFO)¹. Ο ασθενής είτε παίρνει οδηγίες και απόχωρεί, είτε παραπέμπεται για νοσηλευτέςεία.

```

CREATE OR REPLACE VIEW oura_anamenomenwn AS
SELECT

```

d.id_dialogis, d.epipedo,

```

CASE d.epipedo
WHEN 1 THEN 'Άμεσο'
WHEN 2 THEN 'Επείγον'
WHEN 3 THEN 'Επιτακτικό'
WHEN 4 THEN 'Λιγότερο επείγον'
WHEN 5 THEN 'Μη επείγον'
END
AS perigrifi_epipedou,

```

d.wra_afiksis,

```

TIMESTAMPDIFF(MINUTE, d.wra_afiksis, NOW()) AS lepta_anamon_is,

```

d.amka_astheni, d.symptomata

```

FROM dialogistoixeiwn d
WHERE d.apotelesma IS NULL
ORDER BY d.epipedo ASC, d.wra_afiksis ASC;

```

SQL ΕΡΩΤΗΜΑΤΑ ΚΑΙ ΑΝΑΛΥΣΗ

Query 1

Το πρώτο query υπολογίζει το βασικό κόστος (σταθερό ποσό KEN) συν το πρόσθετο κόστος για κάθε μέρα που ξεπέρασε τη ΜΔΝ, ομαδοποιώντας τα αποτελέσματα ανά τμήμα, έτος, KEN κωδικό και ασφαλιστικό φορέα.

```

To index idx_nosileia_tmima_eisodos βοηθά να βρίσκει γρήγορα τις νοσηλευτέςείες ανά

```

τμήμα και να τις ταξινομεί κατά έτος, χωρίς πλήρη σάρωση.

Query 2

Η stored procedure δέχεται ως παράμετρο μια ειδικότητα (π.χ. 'Καρδιολογία') και επιστρέφει όλους τους ιατρούς αυτής της ειδικότητας, δείχνοντας αν είχαν εφημερία το τρέχον έτος (YEAR(CURDATE())) και πόσες ιατρικές πράξεις εκτέλεσαν ως κύριοι χειρουργοί — χρησιμοποιώντας δύο ενδιάμεσα CTE για τις εφημερίες και τις

επεμβάσεις, και στη συνέχεια τα συνδέει με LEFT JOIN ώστε να εμφανίζονται και ιατροί χωρίς εφημερίες ή επεμβάσεις.

```

Χρησιμοποιούνται τα indexes: idx_eidikotita για γρήγορο φιλτράρισμα ειδικότητας,
idx_efimeria_amka_imerominia για εύρεση εφημεριών τρέχοντος έτους και
idx_praxi_kategoria_xeirourgous (κοινό με Query 5) για μέτρηση πράξεων ανά χειρουργό.

```

Query 3

Μετράει νοσηλευτές/είδες ανά ασθενή και τμήμα, φέρνει το κόστος ακόμα και για ανοιχτές νοσηλευτές/είδες (από τον `trexon_kostos_nosileia`), και κρατά μόνο εκείνες >3.

```
To index idx_nosileia_asthenis_tmima χρησιμοποιείται, γιατί ο συνδυασμός  
(amka_astheni, tmima_id) ταιριάζει ακριβώς με το GROUP BY, οπότε η ομαδοποίηση
```

γίνεται χωρίς επιπλέον ταξινόμηση.

Query 4

Εδώ υλοποιήσαμε ένα procedure ώστε να μπορεί το query 4 να τρέξει για κάθε γιατρό. Συγκεκριμένα, βρίσκει όλες τις νοσηλευτές/είδες όπου εμπλέκεται είτε ως εξεταστής είτε ως κύριος χειρουργός, και επιστρέφει τον μέσο όρο των αξιολογήσεων αυτών των νοσηλευτές/είδων στις στήλες «Ποιότητα ιατρικής φροντίδας» και «Συνολική εντύπωση νοσηλευτές/είδας».

Τώρα ας περάσουμε στην ανάλυση με `explain` και `analyze`. Το EXPLAIN μας δείχνει το «σχέδιο» που θα ακολουθήσει ή βάση για να εκτελέσει ένα query — ποια indexes θα χρησιμοποιήσει και πόσες γραμμές θα διαβάσει — χωρίς να το τρέξει στην πραγματικότητα. Το ANALYZE πάει ένα βήμα παραπέρα: εκτελεί κανονικά το query και δίπλα στις εκτιμήσεις δείχνει και τα πραγματικά νούμερα, ώστε να βλέπουμε αν ο optimizer μάντεψε σωστά. Τα FORCE INDEX και IGNORE INDEX είναι «οδηγίες» που δίνουμε στη βάση μέσα στο query: το πρώτο την αναγκάζει να χρησιμοποιήσει συγκεκριμένο index, ενώ το δεύτερο της λέει να αγνοήσει ένα index — χρήσιμο για να δούμε τι θα γινόταν αν αυτό το index δεν υπήρχε, χωρίς να το σβήσουμε στην πραγματικότητα.

Τρέξαμε δύο select (που έχουν την ίδια λογική με το procedure) για έναν συγκεκριμένο γιατρό με `analyze`. Το δεύτερο, χωρίς κάποιο ignore όπου ή βάση χρησιμοποίησε το index `amka_iatrou` και στο πρώτο όπου της είπαμε να αγνοήσει αυτό το index.

Συμπεράσματα: Στην κανονική έκδοση(2η) διάβασε μόνο τις 28 γραμμές που αφορούσαν τον ιατρό. Στην εναλλακτική έκδοση(1η), αναγκάστηκε να διαβάσει 6.279 γραμμές για να βρει τις ίδιες 28 — δηλαδή το 99, 5% της δουλειάς ήταν χαμένη. Η αλλαγή ήταν τόσο μεγάλη που ή βάση άλλαξε και τη σειρά με την οποία ένωσε τους πίνακες (αλλαγή σειράς στα table που βλέπει), κάτι που δείχνει ότι ένα index επηρεάζει όλο το πλάνο, όχι μόνο τον πίνακα του. Σε χρόνο, ή διαφορά ήταν μικρή (0.029s vs 0.001s) γιατί ο πίνακας είναι ακόμα μικρός και χωράει στη μνήμη· με πραγματικά μεγάλα δεδομένα ή διαφορά θα ήταν πολύ πιο εμφανής. Συμπεράσματικά, τα FK indexes που υπάρχουν είναι απαραίτητα για το Q4 και ο optimizer τα επιλέγει σωστά μόνος του — δεν χρειάζεται να του πούμε εμείς τι να κάνει.

Query 5

Συνδέει ιατρούς, ηλικίες και ιατρικές πράξεις κατηγορίας 'Χειρουργική', φιλτράρει ηλικία <35 και τους κατατάσσει κατά πλήθος επεμβάσεων.

```
Αξιοποιήθηκαν τα Indexes idx_ilikia για γρήγορο φίλτρο ηλικίας και  
idx_praxi_kategoria_xeirourgias για ταυτόχρονο φιλτράρισμα κατηγορίας και
```

ομαδοποίηση ανά χειρουργό.

Query 6

Εδώ υλοποιήσαμε ένα procedure ώστε να μπορεί το query 6 να τρέξει για κάθε ασθενή. Συγκεκριμένα, επιστρέφει για τον ασθενή με το δοσμένο AMKA, μία γραμμή ανά νοσηλευτές/είδα με τα στοιχεία της (ID, ημερομηνίες εισόδου/εξόδου, συνολικό κόστος), τους κωδικούς ICD-10 διάγνωσης εισόδου και εξόδου, και τον μέσο όρο των πέντε βαθμολογιών της αξιολόγησης — ταξινομημένη φθίνουσα κατά ημερομηνία εισόδου ώστε να εμφανίζεται πρώτο το πιο πρόσφατο ιστορικό.

Για την σύγκριση, αξίζει να σημειωθεί ότι στην αρχή έτρεξα την κανονική μορφή (χωρίς ignore index) και μετά την αλλαγμένη και ή δεύτερη έβγαινε ψευδώς πιο γρήγορη. Αυτό γινόταν, καθώς ή κανονική έτρεχε πρώτη με cold cache (πληρώνοντας το I/O από δίσκο) και ή β μετά με ήδη ζεσταμένο buffer pool. Οπότε άλλαξα την σειρά και τα αποτελέσματα φαίνονται παρακάτω:

Η κανονική έκδοση χρησιμοποιεί το FK index amka_astheni και διαβάζει μόνο τις 19 γραμμές του ασθενή, ενώ ή εναλλακτική με IGNORE INDEX αναγκάζεται σε full scan και των 1349 νοσηλευτές — 71x περισσότερη δουλειά για το ίδιο αποτέλεσμα. Αυτό απότυπώνεται και στους χρόνους (0.003s vs 0.013s) αλλά και στο r_filtered=1.41% της εναλλακτικής, που σημαίνει ότι το 98.6% των γραμμών που διάβασε τις πέταξε. Συμπέρασματικά, το index στο nosileia.amka_astheni είναι κρίσιμο για queries ιστορικού ασθενή και ή αξία του μεγαλώνει γραμμικά με το μέγεθος του πίνακα.

Query 7

Το query βρίσκει για κάθε δραστική ουσία πόσοι ασθενείς είναι αλλεργικοί σε αυτήν και σε πόσα φάρμακα του φαρμακείου εμπεριέχεται. Η ομαδοποίηση γίνεται με INNER

JOIN στο farmako_drastiki (μόνο ουσίες που υπάρχουν σε τουλάχιστον ένα φάρμακο)

και LEFT JOIN στο allergy, ώστε ουσίες χωρίς καμία καταγεγραμμένη αλλεργία να εμφανίζονται κανονικά με μηδέν αλλεργικούς. Το COUNT(DISTINCT...) και στις δύο πλευρές απότρέπει υπερμέτρηση από το καρτεσιανό γινόμενο των δύο joins.

Δεν χρειάστηκε νέο index — οι ξένες κλειδιά farmako_drastiki.ousia_id και allergy.ousia_id καλύπτονται ήδη από τους αυτόματους indexes που δημιουργεί ή InnoDB στις FK στήλες (το PK της allergy ξεκινάει με amka_astheni, οπότε ξεχωριστός index στο ousia_id υπάρχει για το FK). Η ομαδοποίηση πάνω στο PK της drastiki_ousia δεν απαιτεί κάτι παραπάνω.

Query 8

Το procedure εντοπίζει το προσωπικό ενός τμήματος που, για μια συγκεκριμένη ημερομηνία, δεν έχει αναλάβει εφημερία σε αυτό το τμήμα. Η λογική στηρίζεται σε NOT EXISTS πάνω στο efimeria_proswpiko με ταυτόχρονο φιλτράρισμα ανά amka, tmima και imerominia. Παράλληλα, για κάθε άτομο επιστρέφεται και ή υποκλάση του

(ειδικότητα, βαθμίδα νοσηλευτέζευτή ή ρόλος διοικητικού) μέσω CASE πάνω στο

typos_proswpikou και LEFT JOINs στους τρεις πίνακες υποκλάσης — έτσι κάθε εγγραφή παίρνει την αντίστοιχη πληροφορία χωρίς διπλασιασμό γραμμών.

Το index idx_ef_proswpiko_amka_tmima_imer εξυπηρετεί ακριβώς το NOT EXISTS: το

PK του efimeria_proswpiko ξεκινάει με tmima, οπότε για lookup ανά amka_proswpiko χρειαζόταν index με leading column το AMKA. Ο composite (amka_proswpiko, tmima, imerominia) δίνει direct hit στον έλεγχο ύπαρξης χωρίς σάρωση. Συμπληρωματικά ο

idx_proswpiko_typos βοηθάει όταν χρειαστεί επιπλέον φιλτράρισμα ανά τύπο

προσωπικού.

Query 9

Το query μετράει πόσες μέρες νοσηλευτέζευτήκε κάθε ασθενής ανά έτος και βρίσκει ασθενείς που έχουν τον ίδιο ακριβώς αριθμό ημερών (>15). Το κρίσιμο σημείο είναι τα τρία

SELECT με UNION: ή κανονική περίπτωση (είσοδος και έξοδος στο ίδιο έτος), και οι

δύο cross-year περιπτώσεις για νοσηλευτέζευτες που γεφυρώνουν δύο χρόνια — μία που μετράει τις μέρες έως 31/12 και μία που μετράει τις μέρες από 1/1. Χωρίς αυτή τη διάσπαση, νοσηλευτέζευτες που αλλάζουν χρόνο θα μετρούνταν λάθος.

Το index dx_nosileia_amka_eisodos_eksodos εξυπηρετεί και τις τρεις περιπτώσεις του

UNION — τόσο τον διαχωρισμό κανονικών από cross-year νοσηλευτές (μέσω των ημερομηνιών), όσο και τη γρήγορη ομαδοποίηση ανά ασθενή — χωρίς να χρειαστεί πλήρης σάρωση του πίνακα.

Query 10

Το query βρίσκει τα τρία συχνότερα ζευγάρια δραστικών ουσιών που συνταγογραφούνται μαζί στην ίδια νοσηλευτική ίδιου ασθενή. Πρώτα δημιουργείται το `prescribed_ousies` με τις διακριτές ουσίες ανά νοσηλευτική, και μετά γίνεται self-join του πίνακα αυτού με τον εαυτό του πάνω στα

`(nosileia_id, amka_astheni)`. Η συνθήκη `p1.ousia_id < p2.ousia_id` εξασφαλίζει ότι κάθε ζευγάρι

μετρίεται μία φορά (όχι και ως (A, B) και ως (B, A)) και αποκλείει τα συνεταιρικά ζεύγη με τον εαυτό. Το τελικό `RANK()` με `pair_rank ≤ 3` αφήνει και ισόπαλους στην τρίτη θέση.

Το index `idx_syntagografisi_nosileia_astheni` εξυπηρετεί το self-join: το PK της `syntagografisi`

ξεκινάει με `kod_ema`, οπότε join πάνω στο `(nosileia_id, amka_astheni)` θα απαιτούσε σάρωση. Το νέο composite με leading τα δύο αυτά πεδία επιτρέπει στον optimizer να βρει αμέσως τις άλλες ουσίες της ίδιας νοσηλευτικής/ασθενούς, και επειδή περιλαμβάνει και το `kod_ema`, λειτουργεί ως covering για το επόμενο join στο `farmako_drastiki`.

Query 11

Το query εντοπίζει τους ιατρούς με τις περισσότερες ιατρικές πράξεις ως κύριοι χειρουργοί, και κρατάει όσους έχουν αριθμό πράξεων εντός 5 από το μέγιστο (όχι μόνο τον κορυφαίο). Το `iatroiMeEpemvaseisAirithmo` βγάζει τους ιατρούς με τουλάχιστον μία πράξη, και το `allIatroiMeEpemvaseisArithmo` τους συμπληρώνει με αυτούς που έχουν μηδέν (μέσω LEFT

JOIN στον `iatros` με COALESCE). Το `RANK()` δίνει σειρά κατάταξης, αλλά το τελικό φίλτρο

γίνεται με ρητή σύγκριση `num_iatrikes ≤ max - 5`, ώστε ο "κανόνας του 5" να εκφράζεται σε αριθμό πράξεων και όχι σε θέση.

Δεν χρειάστηκε νέο index — το `iatrikipraxi.amka_kyriou_xeirourgou` είναι FK προς `iatros(amka)` και η InnoDB δημιουργεί αυτόματα index στη στήλη αυτή, ο οποίος καλύπτει πλήρως το GROUP BY `amka_kyriou_xeirourgou`. Ο index από το Q5 (`katigoria, amka_kyriou_xeirourgou`) δεν βοηθάει εδώ γιατί η `katigoria` είναι leading column και το Q11 δεν φιλτράρει σε αυτήν.

Query 12

Το procedure επιστρέφει, για συγκεκριμένη εβδομάδα, τον αριθμό προσωπικού που έχει αναλάβει εφημερία ανά τμήμα, ανά βάρδια και ανά υποκλάση — ειδικότητα για τους ιατρούς, βαθμίδα για τους νοσηλευτές, ρόλος για το διοικητικό. Η ομαδοποίηση δουλεύει με CASE πάνω στο `typos_proswpikou` που επιλέγει την κατάλληλη στήλη από τους τρεις πίνακες υποκλάσης

`(iatros.eidikotita, nosileutis.vathmida_nosileuti, dioikitiko.rolos)`. Το εύρος ορίζεται ως

`[p_week_start, p_week_start + 7 days)` με half-open διάστημα ώστε να μην διπλομετράται ή τελευταία ημέρα.

Το index `idx_ef_proswpiko_imer_tmima_vardia` είναι κρίσιμο γιατί το PK του `efimeria_proswpiko`

ξεκινάει με `tmima` — χωρίς αυτόν, ή σάρωση μιας εβδομάδας θα απαιτούσε πλήρη σάρωση όλων των τμημάτων. Με leading την `imerominia`, ο optimizer κάνει στενό range scan ακριβώς

στις 7 ημέρες. Οι `idx_dioikitiko_rolos` και `idx_nosileutis_vathmida` επιταχύνουν την ομαδοποίηση

ανά υποκλάση όταν αυτή γίνεται μαζική (π.χ. πολλά άτομα διοικητικού/νοσηλευτικού στην ίδια βάρδια).

Query 13

Το query χτίζει αναδρομικά την αλυσίδα εποπτείας κάθε ιατρού: στο βασικό βήμα παίρνει τον άμεσο επόπτη (eripedo = 1), και σε κάθε επόμενη επανάληψη του CTE προχωράει στον επόπτη του επόπτη, αυξάνοντας το eripedo. Έτσι ένας ιατρός με αλυσίδα μήκους 3 εμφανίζεται τρεις φορές, μία για κάθε επίπεδο. Η αναδρομή σταντάει αυτόματα όταν φτάσει σε ιατρό με amka_eroptis IS NULL. Στο τέλος γίνονται joins στο anthropos και για τον εποπτευόμενο και για τον επόπτη ώστε να εμφανίζονται και τα ονόματα.

Δεν χρειάστηκε νέο index – το iatros.amka_eroptis είναι FK με αυτοαναφορά (FOREIGN KEY... REFERENCES iatros(amka)), οπότε η InnoDB δημιουργεί αυτόματα index πάνω του. Αυτός

εξυπηρετεί και το αναδρομικό βήμα (JOIN iatros i ON i.amka = h.amka_eropti, που πάει από επόπτη σε επόπτη μέσω του PK) και τα τελικά joins στο anthropos που χρησιμοποιούν το PK του.

Query 14

Το query εντοπίζει κατηγορίες ICD (πρώτο γράμμα του κωδικού) που είχαν τον ίδιο ακριβώς αριθμό εισαγωγών σε δύο διαδοχικά έτη. Πρώτα κτίζεται ο πίνακας first_table με μετρητή εισαγωγών ανά (κατηγορία, έτος), φιλτράροντας μόνο διαγνώσεις τύπου 'Εισοδος'. Έπειτα γίνεται self-join του ίδιου πίνακα με τον εαυτό του υπό τη συνθήκη f2.year_ = f1.year_ + 1 και ίδιο admissions_count. Το όριο >= 5 αποκλείει στατιστικά αμελητέα ταιριάσματα (π.χ. δύο εισαγωγές που τυχαίνει να επαναληφθούν).

Το index idx_diagnosi_typos_icd εξυπηρετεί το φιλτράρισμα typos_diagnosis = 'Εισοδος' και το

επόμενο join στο nosileia μέσω του nosileia_id. Το PK της diagnosi ξεκινάει με nosileia_id, άρα φίλτρο μόνο σε typos_diagnosis θα απαιτούσε πλήρη σάρωση. Ο νέος composite

(typos_diagnosis, icd, nosileia_id) λειτουργεί επιπλέον ως covering – όλα τα πεδία που

χρησιμοποιεί το query διαβάζονται απευθείας από τον index, χωρίς να χρειαστεί προσπέλαση των rows.

Query 15

Εδώ υλοποιήσαμε ένα view ώστε να μπορεί το query 10 να επιστρέψει όλα τα ζητούμενα στατιστικά. Συγκεκριμένα, το ερωτημα ζητάει ένα “multi layered” grouping πάνω στα επιπεδα επείγοντος τα οποία θα μπορούσαμε να διαχωρίσουμε ως εξής: 1) τα στατιστικά που αφορούν γενικά το κάθε επίπεδο ξεχωριστά (μεσος χρονος εξυπηρέτησης, αριθμος περιστατικων) στατ 2) στατιστικά που αφορούν υποκατηγορίες του κάθε επιπέδου ξεχωριστά (πόσοστό παραπόμπών στα περιστατικά), 3) στατιστικά που αφορούν τις ίδιες τις παραπόμπές (ποσες ανηκουν σε κάθε τμήμα). Έτσι λοιπόν δομησαμε το view με τρία διαφορετικά CTEs. Το πρώτο εφαρμόζει group by eripedo πάνω στο table dialogistoiceiwn όπου κάνουμε την χρήση τριών aggregate συναρτήσεων count, avg, sum προσεχοντας ή ωρα ολοκλήρωσης να μην είναι NULL. Στο δεύτερο CTE (referallsPerDepartment) Στο δεύτερο CTE (referralsperDep) ασχολούμαστε με τις παραπόμπές που οδήγησαν σε νοσηλευτές. Ξεκινάμε πάλι από τον πίνακα dialogistoiceiwn, κρατώντας μόνο τις εγγραφές με apotelesma = 'Παραπόμπη'. Μετά κάνουμε join με τον πίνακα pararobi_gia_nosileia, ώστε κάθε διαλογή να συνδεθεί με τη συγκεκριμένη νοσηλευτές που προκάλεσε, και όχι απλά με τον ασθενή γενικά. Στη συνέχεια, μέσω του πίνακα nosileia, βρίσκουμε το τμήμα στο οποίο έγινε ή εισαγωγή και ενώνουμε με τον πίνακα tmima για να πάρουμε το όνομα του τμήματος. Εδώ το grouping γίνεται ανά eripedo και ανά tmima, ώστε να μετρήσουμε πόσες παραπόμπές κάθε επιπέδου κατέληξαν σε κάθε τμήμα. Για αυτό χρησιμοποιούμε

COUNT(DISTINCT n.nosileia_id).

Τέλος, στο βασικό SELECT ενώνουμε τα δύο CTEs με LEFT JOIN πάνω στο eripedo. Έτσι εμφανίζονται μαζί τα γενικά στατιστικά κάθε επιπέδου και η κατανομή των παραπόμπών του

ανά τμήμα. Το ποσοστό παραπόμπών υπολογίζεται ως: $\text{arithmos_parapobon} * 100.0 / \text{peristatika_ana_epipedo}$ δηλαδή οι παραπόμπές του επιπέδου προς το σύνολο των περιστατικών του ίδιου επιπέδου. Το 100.0 χρησιμοποιείται για να γίνει δεκαδική διαίρεση. Τέλος, ταξινομούμε με βάση το `epipedo` και τις `parapobes_ana_tmima` φθίνουσα, ώστε σε κάθε επίπεδο να φαίνονται πρώτα τα τμήματα με τις περισσότερες παραπόμπές.

Τα indexes `idx_dialogi_epipedo_oloklirosis` και `idx_dialogi_apotelesma_epipedo`

εξυπηρετούν τα δύο σκέλη ξεχωριστά: ο πρώτος (`epipedo`, `wra_oloklirosis`) είναι ευθυγραμμισμένος με το GROUP BY `epipedo` και βοηθάει στο φίλτρο `wra_oloklirosis IS NOT NULL`. ο δεύτερος (`apotelesma`, `epipedo`) εξυπηρετεί τις παραπόμπές, όπου το query φιλτράρει σε `apotelesma = 'Παραπόμπή'` και ομαδοποιεί ανά επίπεδο και τμήμα. Έτσι κάθε CTE παίρνει τον δικό του ad-hoc covering index, χωρίς να αλληλεπιδρούν.